Listas lineares

Universidade Estadual de Mato Grosso do Sul
Bacharelado em Ciência da Computação
Algoritmos e Estruturas de Dados II
Prof. Fabrício Sérgio de Paula

Tópicos

- Introdução
- Alocação sequencial
  - Listas lineares, pilhas e filas, notação polonesa
- Alocação encadeada
  - Listas simplemente encadeadas, pilhas e filas, listas circulares, listas duplamente encadeadas, representação de grafos
- Exercícios

Introdução

- Lista linear (ou tabela): conjunto de $n \geq 0$ nós $\mathcal{L}[1], \mathcal{L}[2], \ldots, \mathcal{L}[n]$ onde:
  - Se $n > 0$, $\mathcal{L}[1]$ é o primeiro nó
  - Para $1 < k \leq n$, o nó $\mathcal{L}[k]$ é precedido por $\mathcal{L}[k-1]$
- Operações frequentes em listas: busca, inclusão e remoção
  - Outras operações: alteração de elemento, combinação de duas listas, ordenação, determinar primeiro e último nó, cardinalidade, etc.

Introdução

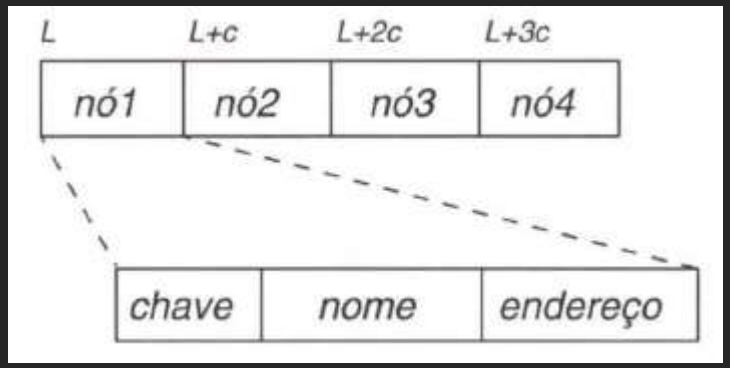
- Tipos de listas lineares: variam de acordo com local de inserções e remoções
  - Pilha: inserções e remoções somente em uma extremidade
  - Fila: inserção em uma extremidade e remoção na outra
  - Deque (double ended queue): inserções e remoções apenas nas extremidades
- Tipo de armazenamento de listas lineares
  - Alocação sequencial: endereços contíguos de memória
  - Alocação encadeada: endereços não contíguos

Alocação sequencial

- Alocação sequencial: nós em posições contíguas de memória
  - Método mais simples
  - Endereço do (j+1)-ésimo nó: endereço do j-ésimo + c
    - c: tamanho (em bytes/palavras) de cada nó
    - Cálculo de endereço: automaticamente pelo compilador
  - Implementação: vetores estáticos/dinâmicos

Alocação sequencial: listas lineares

- Cada nó: composto por campos
  - Campo: característica distinta do elemento da lista
  - Chave: campo identificador único
  - Nós podem estar ordenados ou não pelo campo chave
- Exemplo de lista linear $\mathcal{L}$ com n elementos:



Alocação sequencial: listas lineares

- Algoritmo busca1: busca por um nó na lista
  - Entrada: a chave x do nó a ser buscado
  - Saída: índice do nó cuja chave = x ou 0, caso não exista

```
algoritmo 2.1: Busca de um elemento na lista ℒ
função busca1(x)
    i := 1;   busca1 := 0
    enquanto i ≤ n faça
        se ℒ[i].chave = x então
            busca1 := i                    % chave encontrada
            i := n + 1
        senão i := i + 1                   % pesquisa prossegue
```

Alocação sequencial: listas lineares

- Algoritmo busca: melhoria prática
  - Criação de novo nó com chave procurada na posição n+1

```
algoritmo 2.2: Busca de um elemento na lista ℒ
função busca(x)
    ℒ[n + 1].chave := x;    i := 1
    enquanto ℒ[i].chave ≠ x faça
            i := i + 1
    se i ≠ n + 1 então
            busca := i                    % elemento encontrado
    senão busca := 0                      % elemento não encontrado
```

Alocação sequencial: listas lineares

- Complexidade de pior caso de busca1 e busca: O(n)
  - Na prática: em cada iteração
    - busca1 faz 2 comparações
    - busca faz 1 comparação

Alocação sequencial: listas lineares

- Algoritmo busca-ord: busca em lista ordenada

```
algoritmo 2.3: Busca de um elemento na lista ℒ, ordenada
função busca-ord(x)
    ℒ[n + 1]. chave := x;    i := 1
    enquanto ℒ[i]. chave < x faça
            i := i + 1
    se i = n + 1 ou ℒ[i]. chave ≠ x então
            busca-ord := 0
    senão busca-ord := i
```

- Complexidade de pior caso?

Alocação sequencial: listas lineares

- Algoritmo de busca binária: busca mais eficiente em lista ordenada

```
algoritmo 2.4: Busca binária
função busca-bin(x)
    inf := 1;   sup := n;   busca-bin := 0
    enquanto inf ≤ sup faça
        meio := ⌊(inf + sup)/2⌋          % índice a ser buscado
        se L[meio].chave = x então
            busca-bin := meio            % elemento encontrado
            inf := sup + 1
        senão se L[meio].chave < x então
                inf := meio + 1
            senão sup := meio − 1
```

Alocação sequencial: listas lineares

- Complexidade de pior da busca binária
  - Cada iteração: tempo constante
  - Quantas iterações?
    - Na primeira iteração: tamanho da lista é n
    - Na segunda: tamanho da lista reduzido pela metade
    - Na terceira: tamanho da lista reduzido pela metade
    - ....
    - Última iteração: tamanho da lista é 1
  - São executadas aproximadamente $\log_2 n$ iterações
  - Complexidade: $O(\log_2 n)$
- Versão recursiva da busca binária?

Alocação sequencial: listas lineares

- Operações de inserção e remoção
  - Utilizam procedimento de busca
  - Consideram que a lista não está ordenada
  - Lista tem tamanho n
  - Memória tem M posições disponíveis

Alocação sequencial: listas lineares

- Algoritmo de inserção:

algoritmo 2.5: Inserção de um nó na lista $\mathcal{L}$

se $n < M$ **então**

  se $busca(x) = 0$ **então**

    $\mathcal{L}[n+1] := novo\text{-}valor$

    $n := n + 1$

  senão "elemento já existe na tabela"

senão $overflow$

Alocação sequencial: listas lineares

- Algoritmo de remoção:

```
algoritmo 2.6: Remoção de um nó da lista ℒ
se n ≠ 0 então
        indice := busca(x)
        se indice ≠ 0 então
                valor-recuperado := ℒ[indice]
                para i := indice, n − 1 faça
                        ℒ[i] := ℒ[i + 1]
                n := n − 1
        senão "elemento não se encontra na tabela"
senão underflow
```

Alocação sequencial: listas lineares

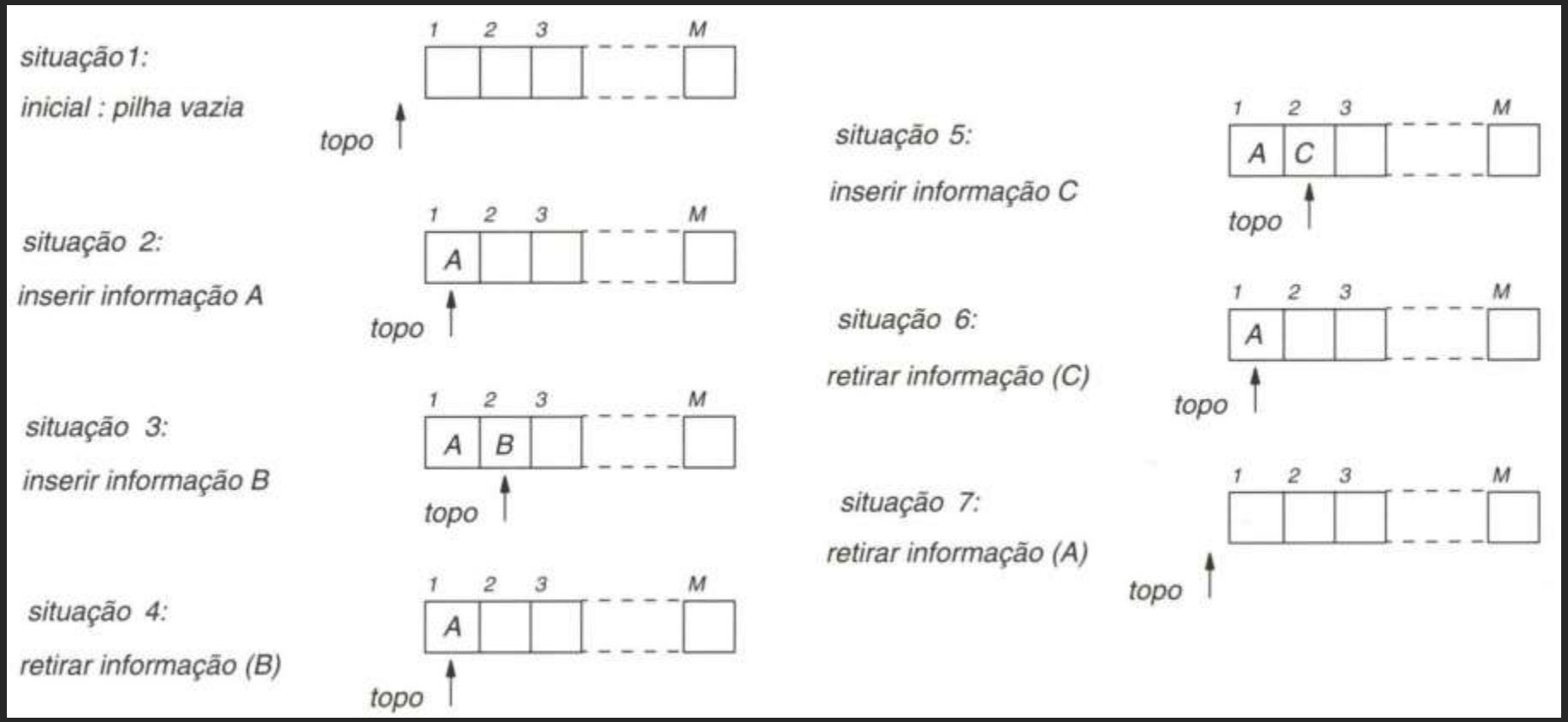
- Complexidade de pior caso da inserção e remoção
  - Dependente da busca: O(n)
  - Remoção: é necessário também movimentar nós
    - O(n) movimentos no pior caso
- E se as listas estivessem ordenadas?
  - Complexidade de pior caso da busca?
    - Busca binária
  - Complexidade da inserção?
  - Complexidade da remoção?

Alocação sequencial: pilhas e filas

- Pilhas, filas e deques: inserções e remoções sem movimentação de nós
  - Alocação sequencial é adequada
- Pilha: estrutura de dados LIFO (*last in first out*)
  - Operações:
    - Inserção/empilha/push
    - Remoção/desempilha/pop
  - Estado da pilha: uso de “ponteiro” (índice) topo
    - Pilha vazia: topo = 0
    - Pilha cheia: topo = M

Alocação sequencial: pilhas e filas

- Operações em uma pilha:



Alocação sequencial: pilhas e filas

- Agoritmos de inserção e remoção na pilha: O(1)

algoritmo 2.7: Inserção na pilha $\mathcal{P}$

se $topo \neq M$ **então**

$\quad topo := topo + 1$

$\quad \mathcal{P}[topo] := novo\text{-}valor$

senão $overflow$

algoritmo 2.8: Remoção da pilha $\mathcal{P}$

se $topo \neq 0$ **então**

$\quad valor\text{-}recuperado := \mathcal{P}[topo]$

$\quad topo := topo - 1$

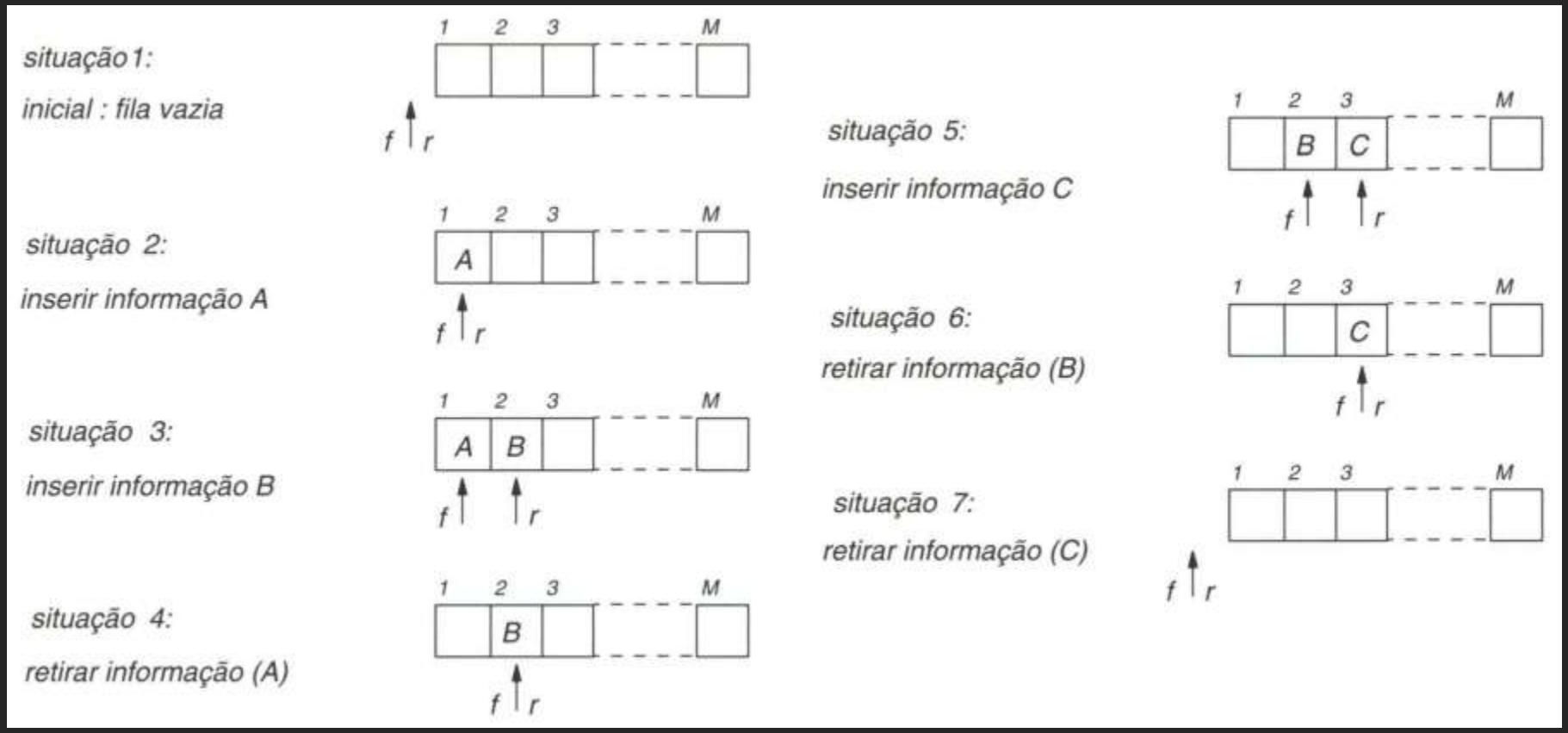
senão $underflow$

Alocação sequencial: pilhas e filas

- Fila: estrutura de dados FIFO (*first in first out*)
  - Operações: inserção e remoção
  - Estado da fila: uso de “ponteiros” início (f) e retaguarda (r)
    - Ponteiros trabalham de forma circular
    - Fila vazia: f = r = 0
    - Fila cheia: (r mod M + 1) = f

Alocação sequencial: pilhas e filas

- Operações em uma fila:



Alocação sequencial: pilhas e filas

- Algoritmos de inserção e remoção na fila: tempo constante

algoritmo 2.9: Inserção na fila $\mathcal{F}$

$prov := r \bmod M + 1$
se $prov \neq f$ **então**
    $r := prov$
    $\mathcal{F}[r] := novo\text{-}valor$
    se $f = 0$ **então**
        $f := 1$
senão $overflow$

algoritmo 2.10: Remoção da fila $\mathcal{F}$

se $f \neq 0$ **então**
    $valor\text{-}recuperado := \mathcal{F}[f]$
    se $f = r$ **então**
        $f := r := 0$
    senão $f := f \bmod M + 1$
senão $underflow$

Aplicação: notação polonesa

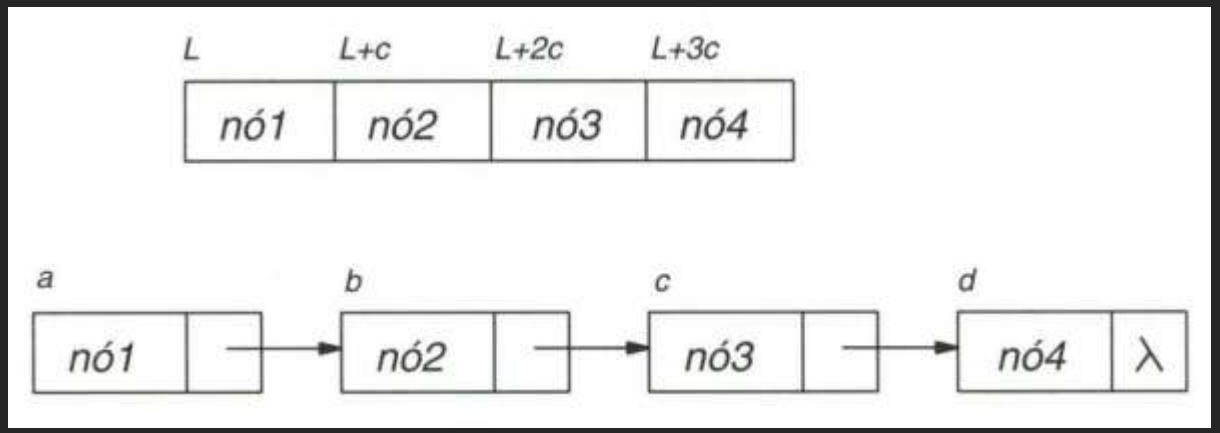
- Representação de expressões aritméticas: compiladores
  - Notação tradicional: A*B – C/D
    - Ambígua: necessidade de regras de prioridade
      - Dificulta computação
  - Notação parentizada: ((A*B) – (C/D))
    - Sempre há par de parênteses a cada par de operandos e operador
  - Notação polonesa: – *AB / CD
    - Operadores imediatamente antes dos operandos: não ambígua
  - Notação polonesa reversa: AB * CD / –
    - Operadores após os operandos: não ambígua
    - Operandos aparecem na mesma ordem da notação tradicional

Aplicação: notação polonesa

```
algoritmo 2.11: Conversão de notações
indexp := 1;   indpol := 0                    % índices das expressões
topo := 0
enquanto indexp ≤ fim faça
        se exp[indexp] é operando então
                indpol := indpol + 1          % passa para a nova lista
                pol[indpol] := exp[indexp]
        senão se exp[indexp] é operador então
                        topo := topo + 1      % coloca na pilha
                        pilha[topo] := exp[indexp]
                senão se exp[indexp] = ")" então
                                se topo ≠ 0 então  % forma a operação
                                        operador := pilha[topo]
                                        topo := topo − 1
                                        indpol := indpol + 1
                                        pol[indpol] := operador
                                senão "expressão errada"
        indexp := indexp + 1
```

Alocação encadeada

- Alocação encadeada (alocação dinâmica): posições de memória alocadas ou desalocadas conforme necessidade
  - Endereços dos nós: não contíguos nem ordenados
    - Nós interligados por ponteiros
    - Ponteiro armazena endereço do próximo nó na lista
    - Ponteiro especial guarda início da lista



Alocação encadeada

- Alocação sequencial x Alocação encadeada
  - Encadeada gasta mais memória: ponteiros
  - Encadeada mais adequada para: juntar listas, separar listas em sublistas, etc.
    - Sem necessidade de puxar/empurrar/copiar elementos
  - Acesso ao k-ésimo elemento da lista
    - Alocação sequencial: imediato
    - Alocação encadeada: percurso até o elemento!

Alocação encadeada: listas lineares simplesmente encadeadas

- Algoritmo para impressão de lista

algoritmo 2.14: Impressão da lista apontada por $ptlista$

$pont := ptlista$

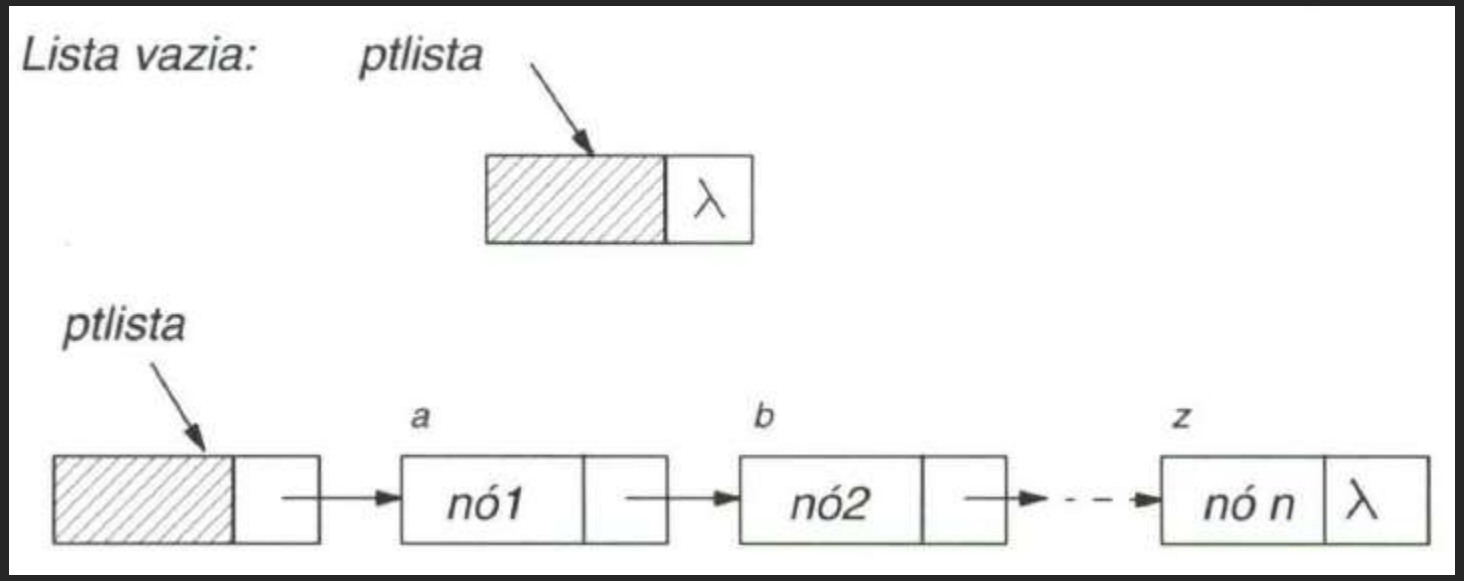
enquanto $pont \neq \lambda$ **faça**

$\quad imprimir(pont \uparrow .info)$

$\quad pont := pont \uparrow .prox$

Alocação encadeada: listas lineares simplesmente encadeadas

- Nó-cabeça: nó especial que nunca é removido
  - Serve para eliminar testes especiais nos algoritmos de inserção e remoção
    - Utilizado nos próximos algoritmos



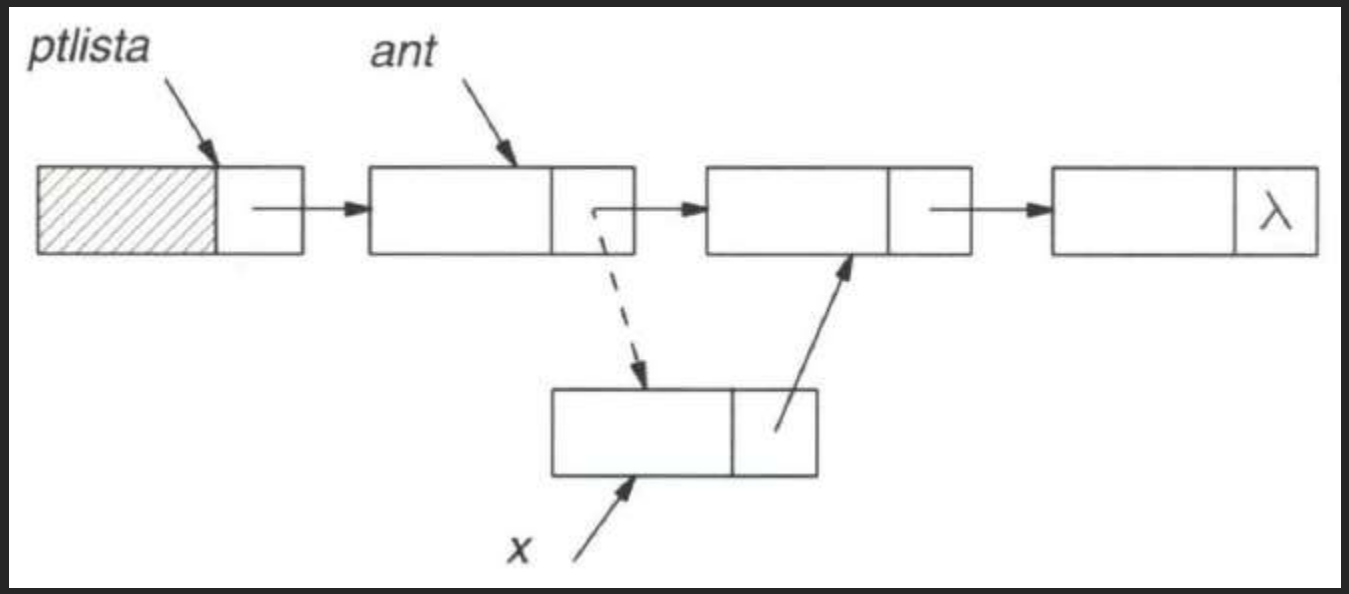
Alocação encadeada: listas lineares simplesmente encadeadas

- Busca em lista ordenada: O(n)

```
algoritmo 2.15: Busca em uma lista ordenada
procedimento busca-enc(x, ant, pont)
    ant := ptlista;    pont := λ
    ptr := ptlista ↑ . prox                    % ptr: ponteiro de percurso
    enquanto ptr ≠ λ faça
        se ptr ↑ . chave < x então
            ant := ptr                         % atualiza ant e ptr
            ptr := ptr ↑ . prox
        senão se ptr ↑ . chave = x então
                pont := ptr                    % chave encontrada
            ptr := λ
```

Alocação encadeada: listas lineares simplesmente encadeadas

- Inserção na lista:



Alocação encadeada: listas lineares simplesmente encadeadas

- Algoritmo de inserção: O(n)

algoritmo 2.16: Inserção de um nó após o nó apontado por *ant*

$busca\text{-}enc(x, ant, pont)$

se $pont = \lambda$ **então**

$\quad ocupar(pt)$ % solicitar nó

$\quad pt \uparrow .info := novo\text{-}valor$ % inicializar nó

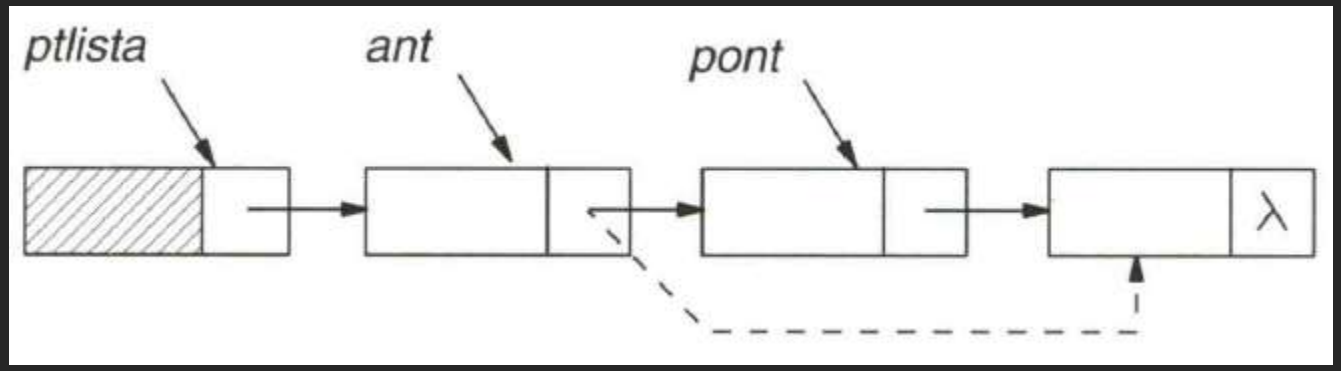
$\quad pt \uparrow .chave := x; \quad pt \uparrow .prox := ant \uparrow .prox$

$\quad ant \uparrow .prox := pt$ % acertar lista

senão "elemento já está na tabela"

Alocação encadeada: listas lineares simplesmente encadeadas

- Remoção na lista:



Alocação encadeada: listas lineares simplesmente encadeadas

- Algoritmo de remoção: O(n)

algoritmo 2.17: Remoção do nó apontado por *pont* na lista

$busca\text{-}enc(x, ant, pont)$

se $pont \neq \lambda$ **então**

$\quad ant \uparrow .prox := pont \uparrow .prox$ % acertar lista

$\quad valor\text{-}recuperado := pont \uparrow .info$ % utilizar nó

$\quad desocupar(pont)$ % devolver nó

senão "nó não se encontra na tabela"

Alocação encadeada: pilhas e filas

- Pilhas e filas: sem nó-cabeça
  - Pilha: ponteiro topo
    - Pilha vazia: topo = λ
  - Lista: ponteiros inicio e fim
    - Fila vazia: inicio = fim = λ

Alocação encadeada: pilhas e filas

- Inserção e remoção na pilha:

```
algoritmo 2.18: Inserção na pilha
ocupar(pt)                                  % solicitar nó
pt ↑ .info := novo-valor                    % inicializar nó
pt ↑ .prox := topo
topo := pt                                  % acertar pilha

algoritmo 2.19: Remoção da pilha
se topo ≠ λ então
        pt := topo                          % acertar pilha
        topo := topo ↑ .prox
        valor-recuperado := pt ↑ .info      % utilizar nó
        desocupar(pt)                       % devolver nó
senão underflow
```

Alocação encadeada: pilhas e filas

- Inserção na fila:

```
algoritmo 2.20: Inserção na fila
ocupar(pt)                                  % solicitar nó
pt ↑ .info := novo-valor                    % inicializar nó
pt ↑ .prox := λ
se fim ≠ λ então                            % acertar fila
        fim ↑ .prox := pt
senão inicio := pt
fim := pt
```

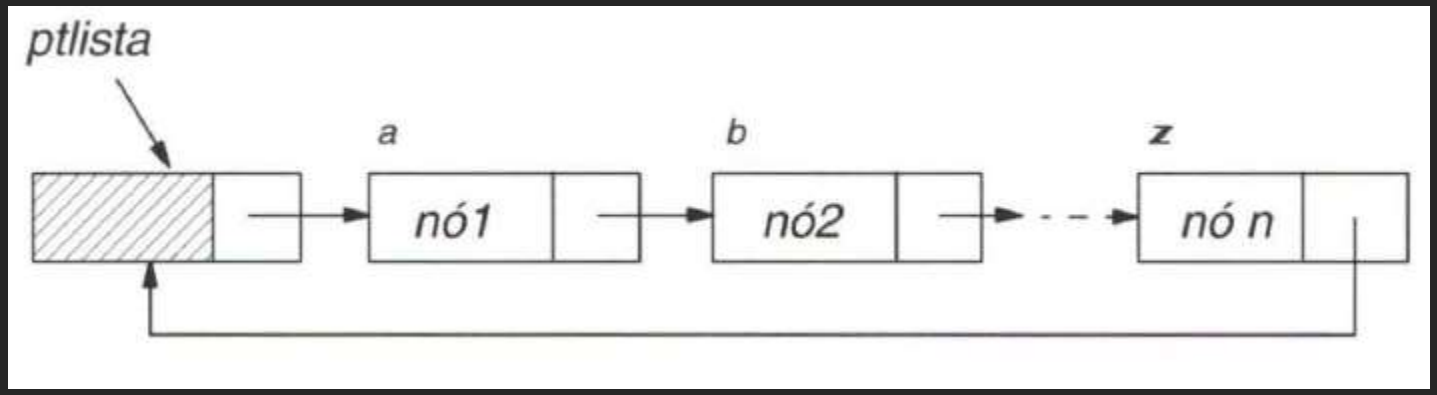
Alocação encadeada: pilhas e filas

- Remoção na fila:

```
algoritmo 2.21: Remoção da fila
se inicio ≠ λ então
        pt := inicio
        inicio := inicio ↑ .prox                    % acertar fila
        se inicio = λ então fim := λ
        valor-recuperado := pt ↑ .info              % utilizar nó
        desocupar(pt)                               % devolver nó
senão underflow
```

Alocação encadeada: listas circulares

- Lista circular: último nó aponta para nó-cabeça
  - Necessidade de readequar: busca, inserção, remoção
    - Não há λ no fim da lista



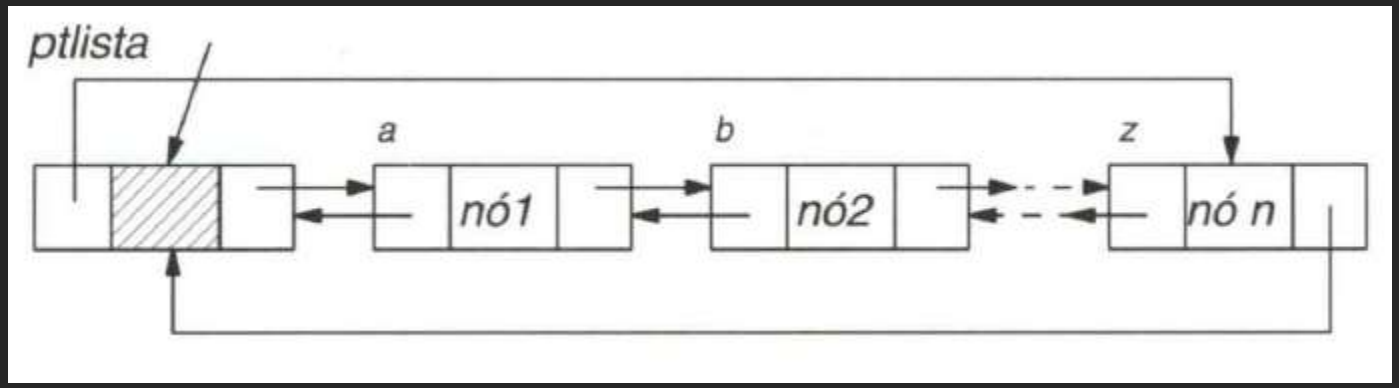
Alocação encadeada: listas circulares

- Busca em lista circular encadeada
  - Complexidade de pior caso?

```
algoritmo 2.23: Busca numa lista circular encadeada ordenada
procedimento busca-cir(x, ant, pont)
    ant := ptlista
    ptlista ↑ .chave := x
    pont := ptlista ↑ .prox
    enquanto pont ↑ .chave < x faça
            ant := pont
            pont := pont ↑ .prox
    se pont ≠ ptlista e pont ↑ .chave = x então
            "chave localizada"
    senão "chave não localizada"
```

Alocação encadeada: listas duplamente encadeadas

- Listas duplamente encadeadas
  - Nem sempre variável ant da busca é suficiente
  - Possibilidade de percorrer a lista nos dois sentidos
    - Nós da lista com dois campos ponteiros: ant e post
  - Podem ser circulares ou não, com nó-cabeça ou não



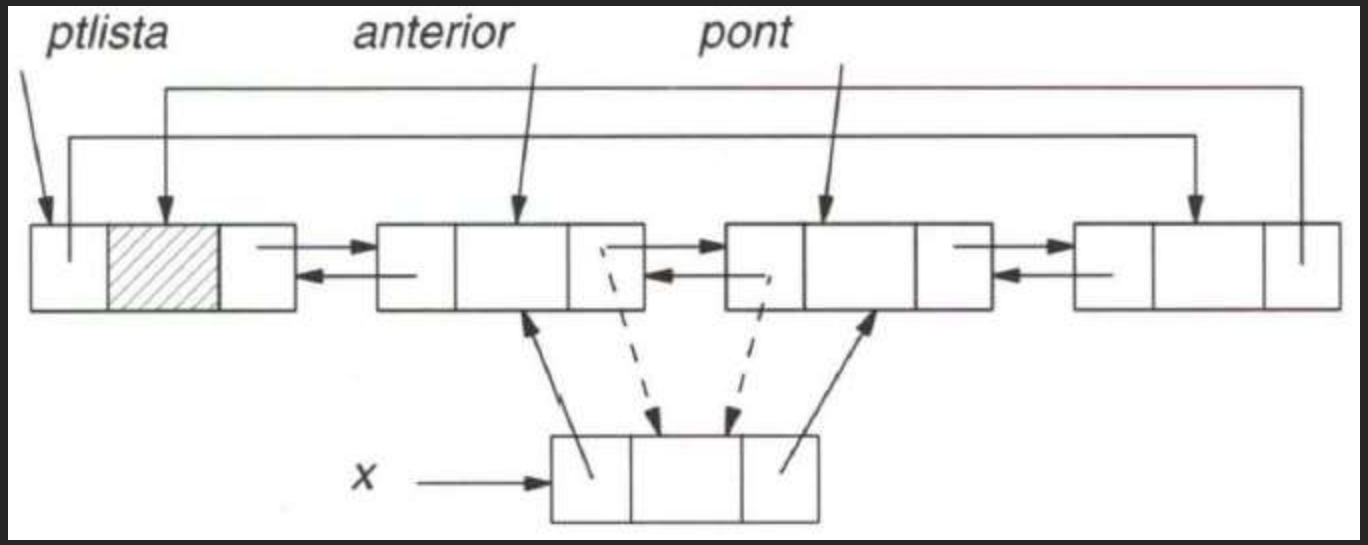
Alocação encadeada: listas duplamente encadeadas

- Busca em lista ordenada:
  - Se x foi encontrado: retorna ponteiro para o nó
  - Caso contrário: ponteiro para o nó que seria consecutivo

```
algoritmo 2.24: Busca em uma lista duplamente encadeada ordenada
função busca-dup(x)
    ultimo := ptlista ↑ .ant
    se x ≤ ultimo ↑ .chave então
        pont := ptlista ↑ .post
        enquanto pont ↑ .chave < x faça
            pont := pont ↑ .post
        busca-dup := pont
    senão busca-dup := ptlista
```

Alocação encadeada: listas duplamente encadeadas

- Inserção:



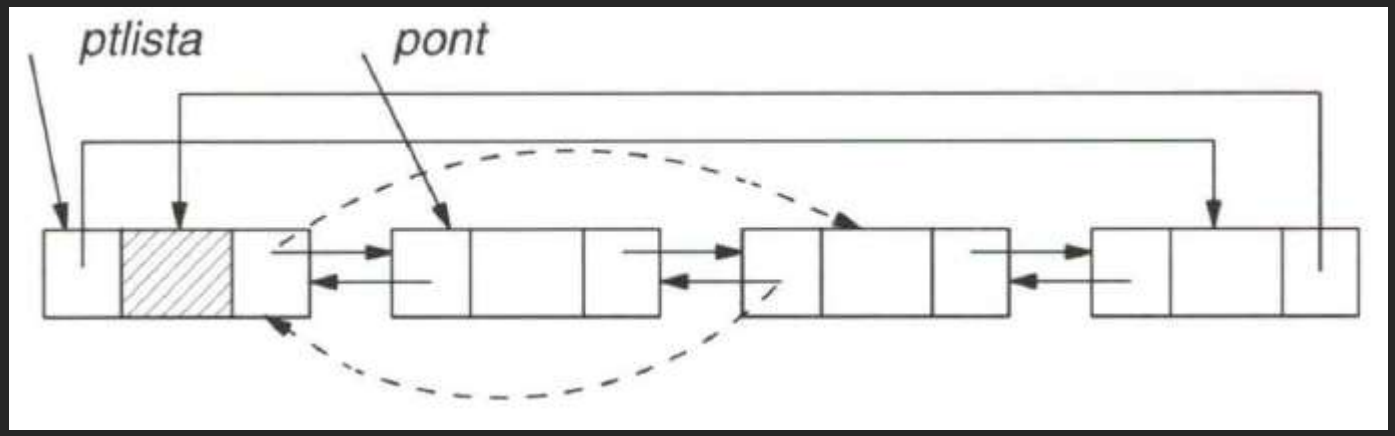
Alocação encadeada: listas duplamente encadeadas

- Algoritmo de inserção: O(n)

```
algoritmo 2.25: Inserção de um nó em uma lista duplamente encadeada
pont := busca-dup(x)
se pont = ptlista ou pont ↑ .chave ≠ x então
        anterior := pont ↑ .ant
        ocupar(pt)                          % solicitar nó
        pt ↑ .info := novo-valor            % inicializar nó
        pt ↑ .chave := x
        pt ↑ .ant := anterior
        pt ↑ .post := pont
        anterior ↑ .post := pt              % acertar lista
        pont ↑ .ant := pt
senão "elemento já se encontra na lista"
```

Alocação encadeada: listas duplamente encadeadas

- Remoção:

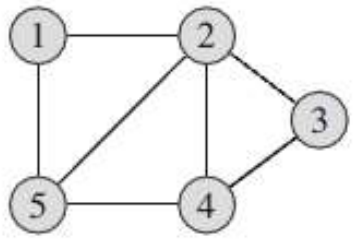


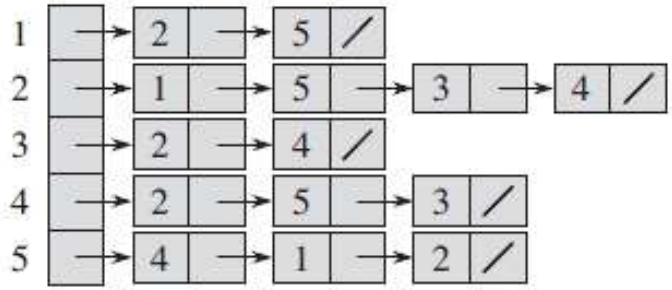
Alocação encadeada: listas duplamente encadeadas

- Algoritmo de remoção: O(n)

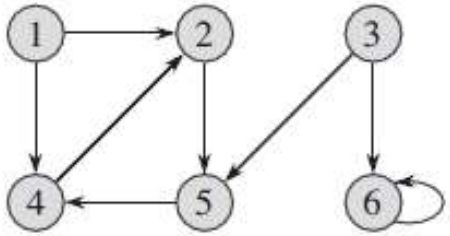
```
algoritmo 2.26: Remoção de um nó em uma lista duplamente encadeada
pont := busca-dup(x)
se pont ≠ ptlista e pont ↑ .chave = x então
        anterior := pont ↑ .ant
        posterior := pont ↑ .post
        anterior ↑ .post := posterior            % acertar lista
        posterior ↑ .ant := anterior
        valor-recuperado := pont ↑ .info         % utilizar nó
        desocupar(pont)                          % devolver nó
senão "elemento não se encontra na lista"
```

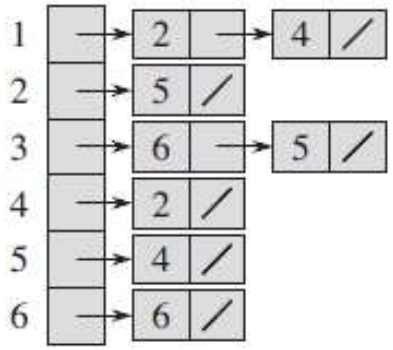
Aplicação: representação de grafos





	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0





	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	0	1	0
3	0	0	0	0	1	1
4	0	1	0	0	0	0
5	0	0	0	1	0	0
6	0	0	0	0	0	1

Exercícios

- Livro Szwarcfiter:
  - 2.1, 2.4, 2.6 a 2.18, 2.20, 2.21